

Rapport des tests Unitaires du Robot CPT1 et 2

NOM DE GROUPE : Asterion Gang

MEMBRE DU GROUPE :

- Félix LANG
- Alexandre JOURDE
- Maxime PAUCHONT
- Tony TRAN
- Timothée TACCA

TABLE DES MATIÈRES

Tâche 1 : LED de la HAL3.1 et LED RGB des 2 feux avant du robot.....	5
1. Présentation du scénario de test.....	5
2. Accès au fichier contenant le programme de test.....	5
3. Schémas matériels.....	5
4. Capture d'écran du programme complet.....	6
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre....	7
6. Références et liens vers les bibliothèques utilisées.....	7
7. Conclusion.....	7
Tâche 2 : LED WS2812 LED.....	8
1. Présentation du scénario de test.....	8
2. Accès au fichier contenant le programme de test.....	8
3. Schémas matériels.....	8
4. Capture d'écran du programme complet.....	9
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	10
6. Références et liens vers les bibliothèques utilisées.....	10
7. Conclusion.....	10
Tâche 3 : Servomoteurs.....	11
1. Présentation du scénario de test.....	11
2. Accès au fichier contenant le programme de test.....	11
3. Schémas matériels.....	11
4. Capture d'écran du programme complet.....	12
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	14
6. Références et liens vers les bibliothèques utilisées.....	14
7. Conclusion.....	14
Tâche 4 : Moteur DC.....	15
1. Présentation du scénario de test.....	15
2. Accès au fichier contenant le programme de test.....	15
3. Schémas matériels.....	15
4. Capture d'écran du programme complet.....	16
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	19
6. Références et liens vers les bibliothèques utilisées.....	19
7. Conclusion.....	19
Tâche 5 : Capteur de mesure ultrason.....	20
1. Présentation du scénario de test.....	20
2. Accès au fichier contenant le programme de test.....	20
3. Schémas matériels.....	20
4. Capture d'écran du programme complet.....	21
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	21
6. Références et liens vers les bibliothèques utilisées.....	21
Tâche 7 : Intégration des fonctions.....	22
1. Présentation du scénario de test.....	22

2. Accès au fichier contenant le programme de test.....	22
3. Capture d'écran du programme complet.....	22
5. Conclusion.....	23
Tâche 6 : Capteur de suivi de ligne.....	24
1. Présentation du scénario de test.....	24
2. Accès au fichier contenant le programme de test.....	24
3. Schémas matériels.....	24
4. Capture d'écran du programme complet.....	25
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	25
6. Références et liens vers les bibliothèques utilisées.....	25
7. Conclusion.....	25
Tâche 8 : Capteur de suivi de lumière.....	26
1. Présentation du scénario de test.....	26
2. Accès au fichier contenant le programme de test.....	26
3. Schémas matériels.....	26
4. Capture d'écran du programme complet.....	27
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	27
6. Références et liens vers les bibliothèques utilisées.....	27
7. Conclusion.....	27
Tâche 9 : Marche avant et arrêt si obstacle.....	28
2. Accès au fichier contenant le programme de test.....	28
3. Schémas matériels.....	28
4. Capture d'écran du programme complet.....	30
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	31
6. Références et liens vers les bibliothèques utilisées.....	31
7. Conclusion.....	31
Tâche 10 : Suivi de source lumineuse et obstacle.....	32
1. Présentation du scénario de test.....	32
2. Accès au fichier contenant le programme de test.....	32
3. Schémas matériels.....	32
4. Capture d'écran du programme complet.....	33
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	34
6. Références et liens vers les bibliothèques utilisées.....	34
7. Conclusion.....	34
Tâche 11 : Suivi de ligne	34
1. Présentation du scénario de test.....	34
2. Accès au fichier contenant le programme de test.....	34
3. Schémas matériels.....	34
4. Capture d'écran du programme complet.....	35
5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre..	37
6. Références et liens vers les bibliothèques utilisées.....	37
7. Conclusion.....	37

Information qui a son importance

Pour les tâches 1 à 10, les captures d'écrans ont été prises avant une modification importante: Sur les photos, le corps du programme est dans un "if __name__ == __main__". Cependant, dans la version la plus récente (celle sur github), le corps du programme est dans un "def run()" afin que l'on puisse utiliser ces programmes dans d'autres programmes. Nous avons tout de même ajouté des :

```
if __name__ == main :  
    run()
```

afin de toujours pouvoir tester les programmes un à un.

Tâche 1 : LED de la HAL3.1 et LED RGB des 2 feux avant du robot

1. Présentation du scénario de test

Cette tâche consiste à commander les neuf LED du robot avec un programme Python. Pour cela, chaque LED est associée à un numéro allant de 1 à 9. Une fonction permet d'allumer ou d'éteindre la LED choisie, tandis qu'une autre permet d'éteindre toutes les LED en même temps.

Le programme principal attend une commande saisie par l'utilisateur. Les commandes de 11 à 19 servent à allumer une LED et les commandes de 21 à 29 servent à l'éteindre. Par exemple, la commande 14 allume la LED 4 et la commande 24 l'éteint. La commande 0 permet d'arrêter le programme.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache1.py

3. Schémas matériels

▪ 3 LED standards sur HAT 3.1:

- La Sortie JP3 est visualisée par LED1: **GPIO 9**
- La Sortie JP4 est visualisée par LED2: **GPIO 25**
- La Sortie JP5 est visualisée par LED3: **GPIO 11**



▪ 2 RGB LED à l'avant du robot:

- A gauche :
left_R : **GPIO00** left_G : **GPIO19** left_B : **GPIO13**
- A droite :
right_R : **GPIO01** right_G : **GPIO05** right_B : **GPIO06**

▪ WS2812 3x RGB LED Module: **GPIO 10**

	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

4. Capture d'écran du programme complet

```
taches > tache1.py > ...
1 import time
2 from gpiozero import LED
3
4 class RobotLEDController:
5     def __init__(self):
6         # Configuration des GPIO pour chaque LED
7         self.LED_CONFIG = {
8             1: {"gpio": 9, "active_high": True},
9             2: {"gpio": 25, "active_high": True},
10            3: {"gpio": 11, "active_high": True},
11
12            4: {"gpio": 0, "active_high": False},
13            5: {"gpio": 19, "active_high": False},
14            6: {"gpio": 13, "active_high": False},
15
16            7: {"gpio": 1, "active_high": False},
17            8: {"gpio": 5, "active_high": False},
18            9: {"gpio": 6, "active_high": False},
19        }
20        # Dictionnaire pour garder les objets LED créés
21        self.leds = {}
22
23        # Initialisation des LEDs
24        def switchSetup(self):
25            for led_id, config in self.LED_CONFIG.items():
26                # Crée chaque LED
27                self.leds[led_id] = LED(config["gpio"], active_high=config["active_high"], initial_value=False)
28
29        # Allumer ou éteindre une LED
30        def switch(self, led_id, status):
31            if led_id not in self.leds:
32                print(f"Erreur : La LED {led_id} n'existe pas.")
33                return
34
35            if status == 1:
36                self.leds[led_id].on() # Allume
37            elif status == 0:
38                self.leds[led_id].off() # Éteint
```

```
39        else:
40            print("Status inconnu")
41
42        # Éteindre toutes les LEDs d'un coup
43        def set_all_switch_off(self):
44            for led_id in self.leds:
45                self.switch(led_id, 0)
46
47
48 if __name__ == "__main__":
49     controller = RobotLEDController()
50     controller.switchSetup() # On configure les LEDs
51     controller.set_all_switch_off() # On éteint tout
52
53     print("Commandes : 11-19 (ON), 21-29 (OFF), 0 (Quitter)")
54     print("-" * 30)
55
56     # Boucle pour lire les commandes de l'utilisateur
57     while True:
58         try:
59             commande = input("Commande : ").strip()
60
61             # Quitter le programme
62             if commande == "0":
63                 controller.set_all_switch_off()
64                 break
```

```

75         if led_id == 0:
76             print("LED invalide")
77             continue
78
79         if action == 1:
80             controller.switch(led_id, 1) # ON
81         elif action == 2:
82             controller.switch(led_id, 0) # OFF
83         else:
84             print("Action inconnue")
85
86     except KeyboardInterrupt:
87         controller.set_all_switch_off()
88         break

```

5. Retour d'expérience

sur les difficultés rencontrées et les solutions mises en œuvre

Aucune difficulté majeure sur cette tâche, cependant il fallait bien lire la documentation technique du robot pour configurer correctement le paramètre active high (à True ou False), afin d'éviter qu'elles ne fonctionnent à l'envers.

6. Références et liens vers les bibliothèques utilisées

<https://gpiozero.readthedocs.io/en/latest/recipes.html>

<https://docs.python.org/3/library/time.html>

7. Conclusion

En conclusion, le programme est entièrement fonctionnel. Toutes les commandes de contrôle des LED sont opérationnelles comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation:

https://youtube.com/shorts/FcMquvPZk1s?is=Mj_BXH31K1I_-DXc

Tâche 2 : LED WS2812 LED

1. Présentation du scénario de test

Cette tâche consiste à concevoir à configurer individuellement les 14 LED du système. Chaque LED est identifiée par un numéro de 1 à 14.

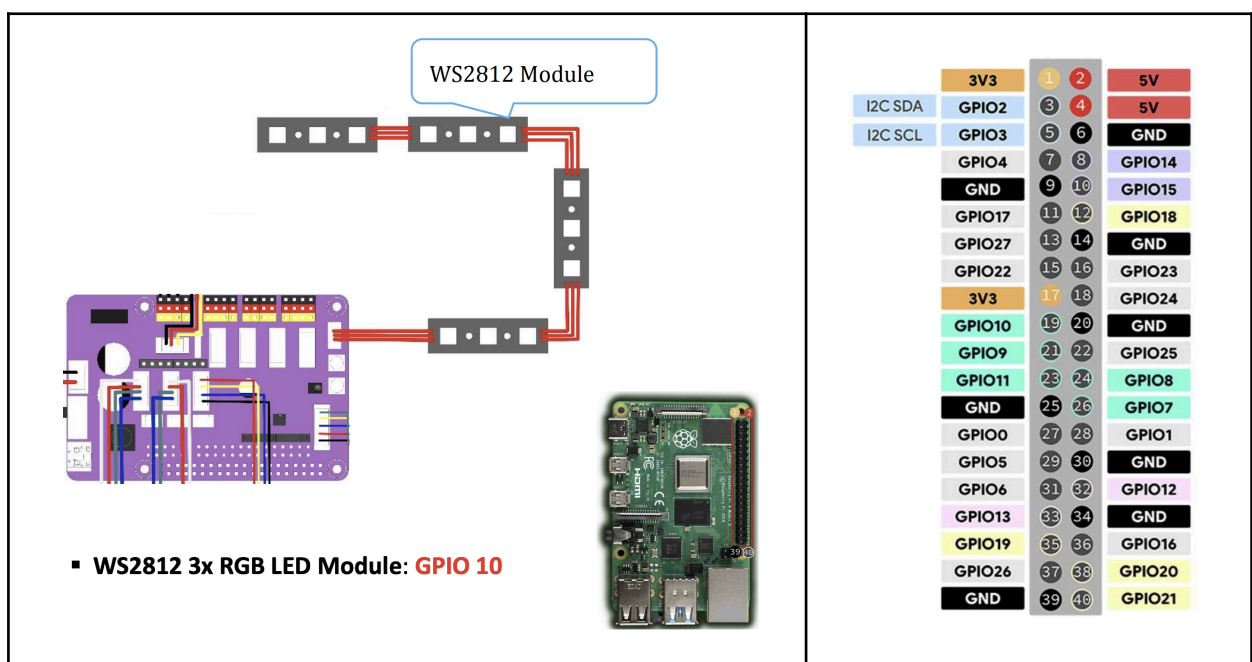
Une fonction prend en charge 5 paramètres : le numéro de la LED, le rouge, le vert, le bleu, ainsi que l'intensité.

L'utilisateur choisit la LED qu'il veut allumer ou éteindre en rentrant une ligne sous la forme : `<index> <r> <g> <b> [brightness]`. Ainsi, la commande `5 255 0 0 255` allumera la LED 5 en Rouge par exemple.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache2.py

3. Schémas matériels



4. Capture d'écran du programme complet

```
taches > tache2.py > LEDController > turn_off_all
1  import numpy
2  import spidev
3
4  class LEDController:
5      def __init__(self, led_count=14):
6          self.led_count = led_count
7          # Tableau pour stocker les valeurs R, G, B de chaque LED
8          self.led_color = [0] * (self.led_count * 3)
9
10         # Configuration de SPI
11         self.spi = spidev.SpiDev()
12         self.spi.open(0, 0)
13         self.spi.mode = 0
14
15         def set_led(self, index, r, g, b, brightness=255):
16             # Vérification de l'index de la LED
17             if not (0 <= index < self.led_count):
18                 print(f"Invalid LED index: {index} (must be 0-{self.led_count - 1})")
19                 return
20
21             # Calcul des couleurs (ordre G, R, B) selon la luminosité
22             self.led_color[index * 3 + 1] = round(r * brightness / 255) # Rouge
23             self.led_color[index * 3 + 0] = round(g * brightness / 255) # Vert
24             self.led_color[index * 3 + 2] = round(b * brightness / 255) # Bleu
25             self.show()
26
27         def show(self):
28             # Conversion du tableau de couleurs en un vecteur plat
29             d = numpy.array(self.led_color).ravel()
30
31             # Encodage des bits pour le protocole des LED (pas codé par nous)
32             tx = numpy.zeros(len(d) * 8, dtype=numpy.uint8)
33             for ibit in range(8):
34                 tx[7 - ibit :: 8] = ((d >> ibit) & 1) * 0x78 + 0x80
35             self.spi.xfer(tx.tolist(), int(8 / 1.25e-6))
36
37         def turn_off_all(self):
38             # Remet toutes les couleurs à 0 et actualise
39             self.led_color = [0] * (self.led_count * 3)
40             self.show()
41
42
43 if __name__ == "__main__":
44     controller = LEDController(led_count=14)
45     controller.turn_off_all()
46     print("Enter: <index> <r> <g> <b> [brightness] - or 'q' to quit")
47
48     # Boucle de lecture des commandes utilisateur
49     while True:
50         try:
51             parts = input("> ").strip().split()
52
53             # Quitter le programme
54             if not parts or parts[0].lower() == "q":
55                 controller.turn_off_all()
56                 break
57
58             # Récupération paramètres entrés
59             index, r, g, b = int(parts[0]), int(parts[1]), int(parts[2]), int(parts[3])
60             brightness = int(parts[4]) if len(parts) >= 5 else 255
61
62             # Application de la commande
63             controller.set_led(index, r, g, b, brightness)
64
65             # Sécurité : éteint tout si on coupe avec Ctrl+C
66             except (KeyboardInterrupt, SystemExit):
67                 controller.turn_off_all()
68                 break
69             except (ValueError, IndexError):
70                 print("<index> <r> <g> <b> [brightness]")
```


5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Au départ, l'utilisation des LED était complexe en se basant uniquement sur la documentation technique. Cependant, après nous être appuyés sur le fichier de test fourni, nous avons pu comprendre rapidement la structure du code pour cette tâche.

6. Références et liens vers les bibliothèques utilisées

<https://numpy.org/doc/>

<https://pypi.org/project/spidev/>

7. Conclusion

En conclusion, le programme est entièrement fonctionnel. Toutes les commandes de contrôle des LED sont opérationnelles comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation:

https://youtube.com/shorts/u_YzuvoEVtk?is=gkaaK-dK5139nmTC

Tâche 3 : Servomoteurs

1. Présentation du scénario de test

L'objectif de cette tâche est de piloter les trois servomoteurs mécaniques du robot AdeepT PiCar-B (canaux CH0, CH1, CH2) à l'aide du contrôleur PCA9685 embarqué sur le HAT V3.1.

La démarche adoptée se déroule en deux étapes :

- Étape 1 : Validation sur un servo libre (CH7) : avant toute interaction avec la mécanique du robot, un quatrième servomoteur non contraint est branché sur CH7. Une séquence automatique (centre → gauche → droite → centre) permet de confirmer le bon fonctionnement du câblage I2C et du code, sans risque de blocage.
- Étape 2 : Commande manuelle interactive : une interface en ligne de commande permet ensuite de piloter individuellement CH0, CH1, CH2 et CH7 en saisissant un numéro de canal et un angle (0° à 180°).

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache3.py

3. Schémas matériels

▪ Moteur de traction DC :

- Sens 1 piloté par **CH14** du **PCA9685**
- Sens 1 piloté par **CH15** du **PCA9685**

▪ Servomoteurs :

- Servomoteur Direction piloté par **CH0** du **PCA9685**
- Servomoteur Tête Dte/Gche activé par **CH1** du **PCA9685**
- Servomoteur Tête Haut/Bas activé par **CH2** du **PCA9685**



	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

4. Capture d'écran du programme complet

```
#!/usr/bin/env python3
# sudo pip3 install adafruit-circuitpython-motor adafruit-circuitpython-pca9685 --break-system-package

import time
from board import SCL, SDA
import busio
from adafruit_motor import servo
from adafruit_pca9685 import PCA9685

# Classe 1 : ServoController
# Gère le PCA9685 et le pilotage de tous les servomoteurs
class ServoController:
    # Plages d'angles sûres par canal
    SAFEANGLES = {
        0: (60, 120), # CH0 - servo mécanique → ±30° autour du centre (90°)
        1: (60, 120), # CH1 - servo mécanique → ±30° autour du centre (90°)
        2: (60, 120), # CH2 - servo mécanique → ±30° autour du centre (90°)
        7: (0, 180), # CH7 - servo libre, pleine plage autorisée
    }

    MIN_PULSE = 500
    MAX_PULSE = 2400

    def __init__(self):
        """Initialise le bus I2C et le contrôleur PCA9685."""
        i2c = busio.I2C(SCL, SDA)
        self.pca = PCA9685(i2c, address=0x5f)
        self.pca.frequency = 50
        print(" ✓ PCA9685 initialisé (I2C 0x5f, 50 Hz)")

    def set_angle(self, servo_id: int, angle: float) -> None:
        min_safe, max_safe = self.SAFEANGLES.get(servo_id, (0, 180))
        safe_angle = max(min_safe, min(max_safe, angle))

        if safe_angle != angle:
            print(f" ⚠ CH{servo_id}: {angle}° limité à {safe_angle}° "
                  f"(plage sûre : {min_safe}°-{max_safe}°)")

        s = servo.Servo(
            self.pca.channels[servo_id],
            min_pulse=self.MIN_PULSE,
            max_pulse=self.MAX_PULSE,
            actuation_range=180
        )
        s.angle = safe_angle

    def center_all(self) -> None:
        """Remet tous les servos configurés à 90° (position centrale)."""
        for ch in self.SAFEANGLES:
            self.set_angle(ch, 90)
            time.sleep(0.1)

    def deinit(self) -> None:
        """Libère proprement le PCA9685."""
        self.pca.deinit()
        print(" PCA9685 libéré.")

# Classe 2 : ServoTester
# Test automatique du servo libre CH7
class ServoTester:
    SEQUENCE = [
        (90, "centre"),
        (45, "gauche"),
        (135, "droite"),
        (90, "retour centre"),
    ]

    def __init__(self, controller: ServoController):
        #controller : ServoController - instance partagée du contrôleur
        self.ctrl = controller

    def run(self) -> None:
        # Lance la séquence de test sur CH7.
        print("\n— Étape 1 : validation CH7 (servo libre) —")
        try:
            for angle, label in self.SEQUENCE:
                print(f" CH7 → {angle}° ({label})")
                self.ctrl.set_angle(7, angle)
                time.sleep(1.0)
            print(" ✓ CH7 OK\n")
```

```

except KeyboardInterrupt:
    print("\nInterruption ☐ retour à 90°.")
    self.ctrl.set_angle(7, 90)

```

```

        parts = saisie.split()
        if len(parts) != 2:
            print("Format attendu : <canal> <angle> (ex: 0 90)")
            continue

        canal = int(parts[0])
        angle = float(parts[1])

        if canal not in self.ctrl.SAFE_ANGLES:
            print(f"Canal {canal} non disponible. "
                  f"Choisir parmi : {list(self.ctrl.SAFE_ANGLES.keys())}")
            continue

        self.ctrl.set_angle(canal, angle)
        print(f"✓ CH{canal} → {angle}°")

    except ValueError:
        print("Valeur invalide. Exemple de saisie correcte : 1 45")
    except KeyboardInterrupt:
        print("\nInterruption ☐ retour à 90°.")
        self.ctrl.center_all()
        break

if __name__ == "__main__":
    print("=== Contrôle Servomoteurs ☐ Robot Adept ===\n")
    controller = ServoController()

    # Étape 1 : test automatique servo libre CH7
    print("Étape 1 : test servo libre CH7")
    tester = ServoTester(controller)
    tester.run()

```

```

# Classe 3 : ServoManual
# Commande manuelle interactive des servomoteurs
class ServoManual:
    CHANNELS = [0, 1, 2, 7]

    def __init__(self, controller: ServoController):
        self.ctrl = controller

    def _afficher_aide(self) -> None:
        # Affiche l'en-tête d'aide au démarrage.
        print("\n" + "=" * 50)
        print(" Commande manuelle des servomoteurs")
        print(" Canaux disponibles : 0, 1, 2 (robot) | 7 (libre)")
        print(" Angles : 0° (gauche) → 90° (centre) → 180° (droite)")
        print(" Saisie : <canal> <angle> | 'q' pour quitter")
        print("=" * 50)

    def run(self) -> None:
        # Lance la boucle de commande manuelle.
        self._afficher_aide()

        print("\nInitialisation à 90° (centre) ...")
        self.ctrl.center_all()
        print("Prêt.\n")

        while True:
            try:
                saisie = input(">>> ").strip().lower()

                if saisie in ("q", "quit", "exit"):
                    print("Sortie ☐ retour à 90° sur tous les canaux.")
                    self.ctrl.center_all()
                    break

                if not saisie:
                    continue

```

```

# Étape 2 : commande manuelle
print("Étape 2 : commande manuelle (CH0, CH1, CH2, CH7)")
manual = ServoManual(controller)
manual.run()

controller.deinit()

```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Le canal CH15 activait les roues arrière

Lors des premiers tests, le servo libre était connecté sur CH15. À chaque commande PWM, les roues arrière du robot s'activaient.

Cause identifiée : selon la documentation AdeepT (Lesson 7, page 4), les canaux CH8–CH15 du PCA9685 sont partagés entre les servomoteurs et les contrôleurs de moteur de traction. Le signal PWM envoyé au servo pilotait simultanément les moteurs.

Pour résoudre ce problème, nous avons déplacé le servo libre sur CH7, qui appartient à la plage CH0–CH7 réservée exclusivement aux servomoteurs. Mise à jour du code en conséquence.

6. Références et liens vers les bibliothèques utilisées

<https://learn.adafruit.com/16-channel-pwm-servo-driver/python-circuitpython>

7. Conclusion

Cette tâche a permis de maîtriser le pilotage de servomoteurs 180° sur le robot AdeepT PiCar-B via le contrôleur PCA9685 et la bibliothèque CircuitPython d'Adafruit.

Vidéo de présentation:

https://youtube.com/shorts/QU9Z_gtbAUw?is=brAWuzQxCDNJ2EKy

Tâche 4 : Moteur DC

1. Présentation du scénario de test

Cette tâche consiste à implémenter une classe qui permet de piloter le moteur DC du Robot. Dans un premier temps, il est possible de faire avancer, reculer ou arrêter le robot. Il est ensuite possible de modifier la vitesse de déplacement. Le changement de vitesse s'effectue en suivant une rampe d'environ 1 seconde. Enfin une commande permet de faire l'étalonnage du servomoteur de direction avec de voir le léger déplacement de ce dernier.

Le moteur DC du robot est commandé dans le terminal. En appuyant sur f, b ou s le robot va avancer, reculer et se stopper. En appuyant sur + ou - le robot va augmenter ou diminuer sa vitesse. Appuyer sur c permet d'accéder à l'étalonnage.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache4.py

3. Schémas matériels

▪ Moteur de traction DC :

- Sens 1 piloté par **CH14** du **PCA9685**
- Sens 1 piloté par **CH15** du **PCA9685**

▪ Servomoteurs :

- Servomoteur Direction piloté par **CH0** du **PCA9685**
- Servomoteur Tête Dte/Gche activé par **CH1** du **PCA9685**
- Servomoteur Tête Haut/Bas activé par **CH2** du **PCA9685**



	3V3	1	2	5V
→ I2C SDA	GPIO2	3	4	5V
→ I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

4. Capture d'écran du programme complet

```
1  #!/usr/bin/env python3
2  import time
3  from board import SCL, SDA
4  import busio
5  from adafruit_pca9685 import PCA9685
6  from adafruit_motor import motor
7
8  # --- Initialisation ---
9  class MotorController:
10     def __init__(self):
11         self.i2c = busio.I2C(SCL, SDA)
12         self.pwm_motor = PCA9685(self.i2c, address=0x5f)
13         self.pwm_motor.frequency = 1000 # 1 kHz pour moteurs DC
14
15         # Configuration des canaux pour les moteurs
16         self.MOTOR_M1_IN1, self.MOTOR_M1_IN2 = 15, 14
17         self.MOTOR_M2_IN1, self.MOTOR_M2_IN2 = 12, 13
18         self.MOTOR_M3_IN1, self.MOTOR_M3_IN2 = 11, 10
19         self.MOTOR_M4_IN1, self.MOTOR_M4_IN2 = 8, 9
20
21         # Création des objets moteur
22         self.motor1 = motor.DCMotor(self.pwm_motor.channels[self.MOTOR_M1_IN1], self.pwm_motor.channels[self.MOTOR_M1_IN2])
23         self.motor2 = motor.DCMotor(self.pwm_motor.channels[self.MOTOR_M2_IN1], self.pwm_motor.channels[self.MOTOR_M2_IN2])
24         self.motor3 = motor.DCMotor(self.pwm_motor.channels[self.MOTOR_M3_IN1], self.pwm_motor.channels[self.MOTOR_M3_IN2])
25         self.motor4 = motor.DCMotor(self.pwm_motor.channels[self.MOTOR_M4_IN1], self.pwm_motor.channels[self.MOTOR_M4_IN2])
26
27         for m in (self.motor1, self.motor2, self.motor3, self.motor4):
28             m.decay_mode = motor.SLOW_DECAY
29
30
31     # Servo de direction sur le même PCA9685, canal 0
32     SERVO_CHANNEL = 0
33     SERVO_LEFT = 3277
34     SERVO_CENTER = 4915
35     SERVO_RIGHT = 6554
36
37     # --- Fonctions internes ---
38     def _map(self, x, in_min, in_max, out_min, out_max):
39         return (x - in_min) / (in_max - in_min) * (out_max - out_min) + out_min
40
41     def _set_all_motors(self, throttle_value):
42         for m in (self.motor1, self.motor2, self.motor3, self.motor4):
43             m.throttle = throttle_value
44
45     def _set_servo(self, duty):
46         self.pwm_motor.channels[SERVO_CHANNEL].duty_cycle = int(duty)
47
48     # --- Fonction de pilotage avec rampe ---
49     # throttle courant global (-1.0 a +1.0)
50     current_throttle = 0.0
```

```

51
52  ✓ def drive_ramp(self, target_throttle, ramp_time=1.0):
53      # target_throttle : valeur signee -1.0 (reculer) a +1.0 (avancer), 0=stop
54      # ramp_time      : duree de la transition en secondes
55      global current_throttle
56      target_throttle = max(-1.0, min(1.0, target_throttle))
57      steps = 100
58      delay = ramp_time / steps
59      for step in range(1, steps + 1):
60          t = self._map(step, 0, steps, current_throttle, target_throttle)
61          self._set_all_motors(t)
62          time.sleep(delay)
63      current_throttle = target_throttle
64      if target_throttle > 0:
65          print(f"[AVANCER] {round(target_throttle * 100)}%")
66      elif target_throttle < 0:
67          print(f"[RECULER] {round(abs(target_throttle) * 100)}%")
68      else:
69          print("[STOP]")
70

```

```

71  # — Etalonnage servo —————
72  ✓ def calibrate_servo(self):
73      print("\n=== ETALONNAGE SERVO ===")
74      print("Le servo est place au centre theorique (duty=4915).")
75      print("Commandes : l=gauche | r=droite | ok=terminer\n")
76      self._set_servo(SERVO_CENTER)
77
78      offset = 0
79      step = 50
80      while True:
81          cmd = input(f" decalage={offset:+d} > ").strip().lower()
82          if cmd == 'ok':
83              print(f"\nDecalage mesure : {offset:+d} (soit duty={SERVO_CENTER + offset})")
84              self._set_servo(SERVO_CENTER) # remet au centre theorique avant de sortir
85              break
86          elif cmd == 'l':
87              offset -= step
88              self._set_servo(SERVO_CENTER + offset)
89          elif cmd == 'r':
90              offset += step
91              self._set_servo(SERVO_CENTER + offset)
92
93  # — Arret propre —————
94  def destroy(self):
95      self._set_all_motors(0)
96      self.pwm_motor.deinit()
97

```



```

98  # — Programme principal : commande manuelle —————
99  if __name__ == '__main__':
100     speed = 25 # vitesse de base en %
101
102     print("=== COMMANDE MANUELLE ===")
103     print(" f/b - avancer/reculer | s - stop")
104     print(" +/- - vitesse +/-10% | c - etalonner servo")
105     print(" q - quitter")
106     print(f" Vitesse de base = {speed}% | Rampe = 1s\n")
107
108     try:
109         while True:
110             cmd = input("Commande > ").strip().lower()
111             if cmd == 'q':
112                 break
113             elif cmd == 'f':
114                 drive_ramp(speed / 100.0)
115             elif cmd == 'b':
116                 drive_ramp(-speed / 100.0)
117             elif cmd == 's':
118                 drive_ramp(0.0)
119             elif cmd == '+':
120                 speed = min(100, speed + 10)
121                 if current_throttle > 0:
122                     drive_ramp(speed / 100.0)
123                 elif current_throttle < 0:
124                     drive_ramp(-speed / 100.0)
125                 print(f" Vitesse -> {speed}%")
126
127             elif cmd == '-':
128                 speed = max(5, speed - 10)
129                 if current_throttle > 0:
130                     drive_ramp(speed / 100.0)
131                 elif current_throttle < 0:
132                     drive_ramp(-speed / 100.0)
133                 print(f" Vitesse -> {speed}%")
134             elif cmd == 'c':
135                 calibrate_servo()
136             else:
137                 print(" Commande inconnue.")
138         except KeyboardInterrupt:
139             pass
140         finally:
141             destroy()

```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Pas de réel difficulté rencontrée durant cette tâche, si ce n'est la compréhension de ce qui nous était demandé. La partie sur l'étalonnage n'était pas claire en se référant uniquement à l'objectif. Pour les autres objectifs, la documentation et le fichier test étaient suffisants pour la compréhension.

6. Références et liens vers les bibliothèques utilisées

<https://docs.circuitpython.org/projects/motor/en/latest/>

<https://docs.circuitpython.org/projects/pca9685/en/latest/>

<https://docs.circuitpython.org/projects/blinka/en/latest/>

7. Conclusion

En conclusion, le programme est entièrement fonctionnel. Toutes les commandes de contrôle du moteur DC sont opérationnelles comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation:

<https://www.youtube.com/shorts/qTOyZzVXBf4>

Tâche 5 : Capteur de mesure ultrason

1. Présentation du scénario de test

Cette tâche consiste à implémenter une classe de mesure de distance entre le robot et un obstacle en utilisant le capteur ultrason du robot. Le programme mesure la distance en mm et retourne la distance dans le terminal.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache5.py

3. Schémas matériels

▪ Tracking module:

- Left line IR sensor is connected to **GPIO 19**
- Middle line IR sensor is connected to **GPIO 16**
- Right line IR sensor is connected to **GPIO 20**

▪ Ultrasonic sensor:

- Trig pin of the ultrasonic sensor: **GPIO 11**
- Echo pin of the ultrasonic sensor: **GPIO 8**

▪ I2C COM Protocol (**GPIO 2** and **GPIO 3**)

- Servo motors are controlled by the PCA9685 chip:
Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x40**
- MPU6050 uses I2C COM protocol with **address 0x68**
- OLED screen uses I2C COM protocol with **address 0x3C**



	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

La demande d'envoi de l'onde ultrasonore est faite par le pin Trig (GPIO 11), et le pin Echo (GPIO 8) mesure le temps mis par l'onde ultrasonore pour partir et revenir, et ce temps peut ensuite être utilisé pour calculer la distance avec l'obstacle.

4. Capture d'écran du programme complet

```
from gpiozero import DistanceSensor
from time import sleep

# Initialisation de la classe Distance pour le capteur à ultrasons
class Distance:

    def __init__(self):
        self.Tr = 23
        self.Ec = 24
        # Create a DistanceSensor object with a 2 meter maximum detection range.
        self.sensor = DistanceSensor(echo=self.Ec, trigger=self.Tr, max_distance=2)

    def checkdist(self):
        return self.sensor.distance * 1000 # Convert meters to millimeters.

if __name__ == "__main__":
    # Instantiate the Distance helper and print readings continuously.
    distance_sensor = Distance()
    while True:
        distance = distance_sensor.checkdist()
        print("%.2f mm" % distance)
        sleep(0.05)
```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Pas vraiment de difficulté rencontrées. En prenant l'exemple 07_ultra.py, il suffisait de changer l'unité de cm à mm et d'en faire une classe.

6. Références et liens vers les bibliothèques utilisées

<https://gpiozero.readthedocs.io/en/latest/recipes.html>

<https://docs.python.org/3/library/time.html>

7. Conclusion

Cette tâche a permis d'implémenter une classe de mesure de distance par capteur ultrason sur le robot. L'adaptation de l'exemple 07_ultra.py s'est faite en convertissant l'unité de cm en mm et en restructurant le code en classe. Le pin Trig (GPIO 11) émet l'impulsion et le pin Echo (GPIO 8) mesure le temps de retour, permettant de calculer la distance en millimètres. La tâche n'a présenté aucune difficulté particulière et constitue une base utile pour la détection d'obstacles dans les prochaines tâches.

Voici la vidéo de présentation :

https://youtube.com/shorts/hRBitakqR_U?is=JrKBS9cJyXFcdicf

Tâche 7 : Intégration des fonctions

1. Présentation du scénario de test

Cette tâche consiste à réaliser l'intégration globale du système en vérifiant le fonctionnement simultané des différents modules logiciels développés précédemment.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache7.py

3. Capture d'écran du programme complet

```
1  import tache1 as t1
2  import tache2 as t2
3  import tache3 as t3
4  import tache4 as t4
5  import tache5 as t5
6  import tache6 as t6
7
8  robot_led_controller = t1.RobotLEDController()
9  led_controller = t2.LEDController(led_count=14)
10 servo_controller = t3.ServoController()
11 servo_sequence = t3.ServoTester(servo_controller)
12 servo_manual = t3.ServoManual(servo_controller)
13 motor_controller = t4.MotorController()
14 line_follower = None # Don't initialize - will be created fresh for each task 6 run
15
16
17 if __name__ == "__main__":
18     action = input("Entrez les numeros de tache a executer separes d'un espace : ").strip().split()
19     for i in range(len(action)):
20         try:
21             if action[i] == '1':
22                 t1.run()
23             elif action[i] == '2':
24                 t2.run()
25             elif action[i] == '3':
26                 t3.run()
27             elif action[i] == '4':
28                 t4.run()
29             elif action[i] == '5':
30                 t5.run()
31             elif action[i] == '6':
32                 t6.run()
33         except KeyboardInterrupt:
34             print("\nTask interrupted by user")
35             break
36         except Exception as e:
37             print(f"Error running task {action[i]}: {e}")
38             # Cleanup GPIO resources if tasks have cleanup methods
39             try:
40                 if hasattr(t5, 'distance_sensor') and hasattr(t5.distance_sensor, 'cleanup'):
41                     t5.distance_sensor.cleanup()
42             except:
43                 pass
44
```

4. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Comme la plupart des tâches avaient déjà été implémentées sous forme de classes, il ne nous restait plus qu'à convertir la tâche 1 en classe et à créer un fichier de test pour exécuter la tâche souhaitée

5. Conclusion

En conclusion, le programme est entièrement fonctionnel. et nous pouvons exécuter n'importe quelle tâche.

Tâche 6 : Capteur de suivi de ligne

1. Présentation du scénario de test

Cette tâche consiste à créer une classe de suivi de ligne à trois capteurs infrarouges pour permettre au robot de se repérer au sol. Le programme détermine si la surface détectée réfléchit la lumière (1) ou non (0)

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache6.py

3. Schémas matériels

Tracking module:

- Left line IR sensor is connected to **GPIO 19**
- Middle line IR sensor is connected to **GPIO 16**
- Right line IR sensor is connected to **GPIO 20**

Ultrasonic sensor:

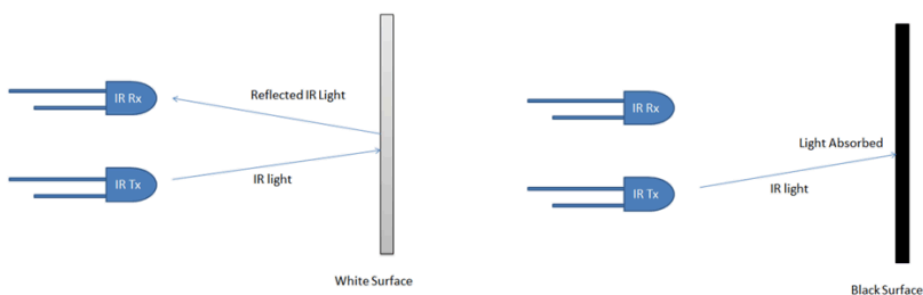
- Trig pin of the ultrasonic sensor: **GPIO 11**
- Echo pin of the ultrasonic sensor: **GPIO 8**

I2C COM Protocol (**GPIO 2** and **GPIO 3**)

- Servo motors are controlled by the PCA9685 chip:
Interfacing to Raspberry Pi is done using I2C COM protocol with address 0x40
- MPU6050 uses I2C COM protocol with address 0x68
- OLED screen uses I2C COM protocol with address 0x3C



3V3	GPIO2	3	2	5V
I2C SDA	GPIO3	4	4	5V
I2C SCL	GPIO4	5	6	GND
GND	GPIO17	7	8	GPIO14
GPIO17	GPIO27	9	16	GPIO15
GPIO27	GPIO22	11	12	GPIO18
GPIO22	3V3	13	14	GND
GPIO10	GPIO9	15	16	GPIO23
GPIO9	GPIO11	17	18	GPIO24
GPIO11	GND	19	20	GND
GND	GPIO0	21	22	GPIO25
GPIO0	GPIO5	23	24	GPIO8
GPIO5	GPIO6	25	26	GPIO7
GPIO6	GPIO13	27	28	GPIO1
GPIO13	GPIO19	29	30	GND
GPIO19	GPIO26	31	32	GPIO12
GPIO26	GND	33	34	GND
GND		35	36	GPIO16
		37	38	GPIO20
		39	40	GPIO21



Si un sensor (connecté au pin GPIO 19, 16 ou 20) émet de la lumière qui se réfléchit sur la surface, il renverra 1 comme valeur, sinon 0.

4. Capture d'écran du programme complet

```
import time
import argparse
from gpiozero import InputDevice

# Initialisation de la classe LineFollower pour les capteurs de suivi de ligne
class LineFollower:
    def __init__(self):
        self.line_pin_left = 22
        self.line_pin_middle = 27
        self.line_pin_right = 17

        self.left = InputDevice(pin=self.line_pin_right)
        self.middle = InputDevice(pin=self.line_pin_middle)
        self.right = InputDevice(pin=self.line_pin_left)

    def run(self):
        status_right = self.right.value
        status_middle = self.middle.value
        status_left = self.left.value
        print('left: %d middle: %d right: %d' %(status_right,status_middle,status_left))

if __name__ == '__main__':
    try:
        line_follower = LineFollower()
        while 1:
            line_follower.run()
            time.sleep(0.3)
    except KeyboardInterrupt:
        pass
```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Nous avons pris l'exemple 08_lineTracking.py, et nous en avons fait une classe, donc pas de difficultés rencontrées.

6. Références et liens vers les bibliothèques utilisées

<https://gpiozero.readthedocs.io/en/latest/recipes.html>

<https://docs.python.org/3/library/time.html>

<https://docs.python.org/fr/3/howto/argparse.html>

7. Conclusion

Cette tâche a permis d'implémenter une classe de suivi de ligne utilisant 3 capteurs infrarouges connectés aux pins GPIO 19, 16 et 20. En s'appuyant sur l'exemple 08_lineTracking.py, l'adaptation s'est limitée à restructurer le code en classe. Chaque capteur retourne 1 si la surface réfléchit la lumière, 0 sinon, fournissant ainsi une information de position utile pour guider le robot le long d'une ligne au sol, ce qui sera utile plus tard.

Voici la vidéo de présentation:

https://youtube.com/shorts/q8c2SPDmTh8?is=3CjGaxkw6VgQud_p

Tâche 8 : Capteur de suivi de lumière

1. Présentation du scénario de test

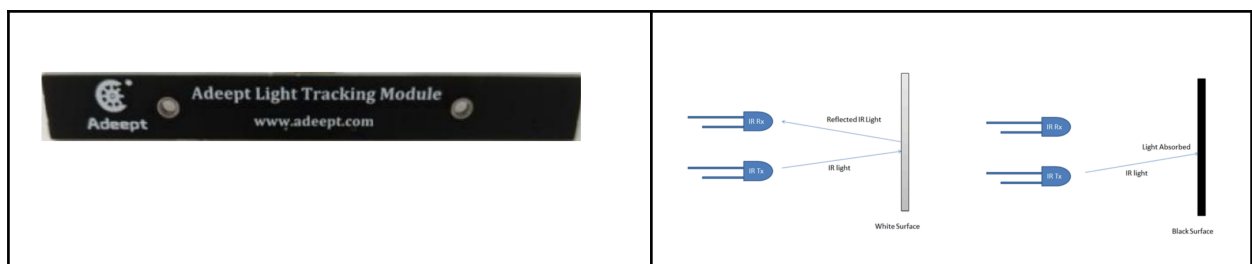
Cette tâche consiste à implémenter le capteur de suivi de lumière. Le programme traque la différence de réflexion de la lumière (une surface blanche réfléchit intensément le signal, tandis qu'une surface noire l'absorbe)

Par exemple l'approche d'un obstacle comme un doigt entraîne une baisse de valeur sur le capteur droit et une hausse sur le capteur gauche. Ces variations numériques sont affichées en temps réel sur le terminal

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/examples/09_lightTracking.py

3. Schémas matériels



Si un sensor émet de la lumière qui se réfléchit sur la surface, il renverra 1 comme valeur, sinon 0.

4. Capture d'écran du programme complet

```
examples > 09_lightTracking.py > ...
1  import time
2  import smbus
3
4
5  class ADS7830(object):
6      def __init__(self):
7          self.cmd = 0x84
8          self.bus=smbus.SMBus(1)
9          self.address = 0x48 # 0x48 is the default i2c address for ADS7830 Module.
10
11      def analogRead(self, chn): # ADS7830 has 8 ADC input pins, chn:0,1,2,3,4,5,6,7
12          value = self.bus.read_byte_data(self.address, self.cmd|(((chn<<2 | chn>>1)&0x07)<<4))
13          return value
14
15  if __name__ == "__main__":
16      adc = ADS7830()
17      while True:
18          adc_value = adc.analogRead(1)
19          print(f"Light Tracking Value: {adc_value}")
20          time.sleep(0.5)
```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Nous n'avons rencontré aucune difficulté sur cette tâche car il suffisait d'exécuter le fichier de test fourni, et la lecture de la documentation permet de comprendre le principe de fonctionnement de ces capteurs de lumière.

6. Références et liens vers les bibliothèques utilisées

<https://docs.python.org/3/library/time.html>

<https://smbus.org/specs/>

7. Conclusion

En conclusion, le programme est entièrement fonctionnel. Le suivi de lumière et l'affichage des variations en temps réel sur le terminal sont tout à fait opérationnels, comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation:

https://youtube.com/shorts/3NBemZKaw8A?is=2U2y_ZtDjnl3P-0a

Tâche 9 : Marche avant et arrêt si obstacle

1. Présentation du scénario de test

L'objectif de cette tâche est de fournir un programme permettant au robot d'avancer en marche avant, puis de s'arrêter automatiquement lorsqu'un obstacle est détecté à moins de 20 cm. Un système de feux de détresse est activé lors de l'arrêt d'urgence.

Le scénario se déroule en quatre étapes successives :

- Départ en marche avant déclenché par l'envoi du caractère 'M'.
- Arrêt immédiat manuel possible à tout moment par l'envoi du caractère 'A'.
- Si un obstacle est détecté à moins de 20 cm devant le robot : arrêt automatique et activation des feux de détresse (phares GPIO + LEDs WS2812 orange clignotants).
- Un nouvel envoi de 'M' redémarre le robot et désactive les feux de détresse.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/tree/master/taches/tache9.py

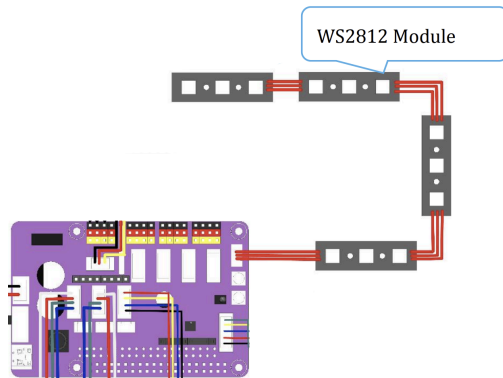
3. Schémas matériels

▪ **Moteur de traction DC :**

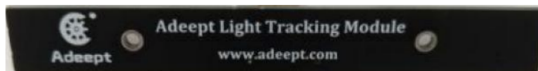
- Sens 1 piloté par **CH14** du **PCA9685**
- Sens 1 piloté par **CH15** du **PCA9685**

▪ **Servomoteurs :**

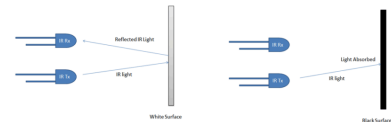
- Servomoteur Direction piloté par **CH0** du **PCA9685**
- Servomoteur Tête Dte/Gche activé par **CH1** du **PCA9685**
- Servomoteur Tête Haut/Bas activé par **CH2** du **PCA9685**



	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21



Si un sensor émet de la lumière qui se réfléchit sur la surface, il renverra 1 comme valeur, sinon 0.



▪ **Tracking module:**

- Left line IR sensor is connected to **GPIO 19**
- Middle line IR sensor is connected to **GPIO 16**
- Right line IR sensor is connected to **GPIO 20**

▪ **Ultrasonic sensor:**

- Trig pin of the ultrasonic sensor: **GPIO 11**
- Echo pin of the ultrasonic sensor: **GPIO 8**



La demande d'envoi de l'onde ultrasonore est faite par le pin Trig (GPIO 11), et le pin Echo (GPIO 8) mesure le temps mis par l'onde ultrasonore pour partir et revenir, et ce temps peut ensuite être utilisé pour calculer la distance avec l'obstacle.

	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

4. Capture d'écran du programme complet

```
1  #!/usr/bin/env python3
2  import time
3  import threading
4
5  import tache1 as t1
6  import tache2 as t2
7  import tache4 as t4
8  import tache5 as t5
9
10 class RobotController:
11
12     VITESSE_MARCHE = 0.25
13     DIST_OBSTACLE_MM = 200.0
14     PERIODE_CAPTEUR = 0.05
15
16     def __init__(self):
17         self.leds_gpio = t1.RobotLEDController()
18         self.leds_gpio.switchSetup()
19         self.leds_gpio.set_all_switch_off()
20
21         self.leds_ws = t2.LEDController(led_count=14)
22         self.leds_ws.turn_off_all()
23
24         self.mc = t4.MotorController()
25         self.capteur = t5.Distance()
26
27         self.en_marche = False
28         self.feux_actifs = False
29         self.stop_feux = threading.Event()
30
31     # — Feux —
32
```

```
32
33     def _clignoter_feux(self):
34         while not self.stop_feux.is_set():
35             for i in range(1, 10):
36                 self.leds_gpio.switch(i, 1)
37             for i in range(14):
38                 self.leds_ws.set_led(i, 255, 80, 0)
39                 time.sleep(0.4)
40             self.leds_gpio.set_all_switch_off()
41             self.leds_ws.turn_off_all()
42             time.sleep(0.4)
43
44     def activer_feux(self):
45         if self.feux_actifs:
46             return
47         self.feux_actifs = True
48         self.stop_feux.clear()
49         threading.Thread(target=self._clignoter_feux, daemon=True).start()
50         print("[FEUX] Feux de detresse ON")
51
52     def desactiver_feux(self):
53         if not self.feux_actifs:
54             return
55         self.stop_feux.set()
56         self.feux_actifs = False
57         self.leds_gpio.set_all_switch_off()
58         self.leds_ws.turn_off_all()
59         print("[FEUX] Feux de detresse OFF")
60
61     # — Moteur —

```

```

98     try:
99         while True:
100             cmd = input("Commande > ").strip()
101             if cmd == 'M':
102                 self.demarrer()
103             elif cmd in ('A', 'a'):
104                 self.arreter()
105                 self.desactiver_feux()
106             elif cmd == 'q':
107                 break
108             else:
109                 print("  Commandes : M | A | q")
110
111         except KeyboardInterrupt:
112             print("\nFin de programme par Ctrl-C")
113
114         finally:
115             self.arreter()
116             self.desactiver_feux()
117             self.mc.destroy()
118             print("Nettoyage final realise")
119
120
121 if __name__ == "__main__":
122     RobotController().run()

```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

Cette tâche n'est qu'un assemblage de tâches déjà réalisées et n'a présenté de réel problème

6. Références et liens vers les bibliothèques utilisées

<https://docs.python.org/3/library/threading.html>

<https://docs.circuitpython.org/projects/motor>

<https://smbus.org/specs/>

7. Conclusion

En conclusion, le programme est entièrement fonctionnel. Le suivi de lumière et l'affichage des variations en temps réel sur le terminal sont tout à fait opérationnels, comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation :

<https://youtube.com/shorts/w5B2Wq78nd4?is=o7z1zUV7TMeKTKbs>

Tâche 10 : Suivi de source lumineuse et obstacle

1. Présentation du scénario de test

L'objectif de cette tâche est de tester le capteur de suivi de lumière du robot. Ce capteur mesure la lumière réfléchiée par une surface, une surface blanche renvoie beaucoup de lumière, tandis qu'une surface noire l'absorbe.

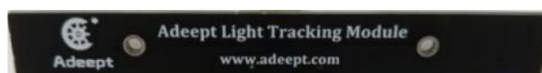
Lorsque l'on approche un obstacle, la luminosité change. Cela fait baisser la valeur du capteur droit et monter celle du capteur gauche. Ces variations s'affichent en direct sur l'écran du terminal pour vérifier que le robot réagit correctement.

Le robot démarre en marche avant quand on envoie 'M' et s'arrête instantanément si on envoie 'A' ou 'a'. En mode marche, il suit une source lumineuse grâce à ses capteurs, tout en mesurant la distance devant lui. Si un obstacle est à moins de 20 cm, il s'arrête avec les feux de détresse (1s), recule avec des « Bip Bip » (30 cm), puis fait une pause (2s) avant de reprendre le suivi.

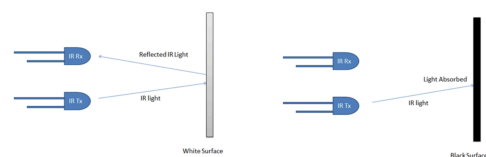
2. Accès au fichier contenant le programme de test

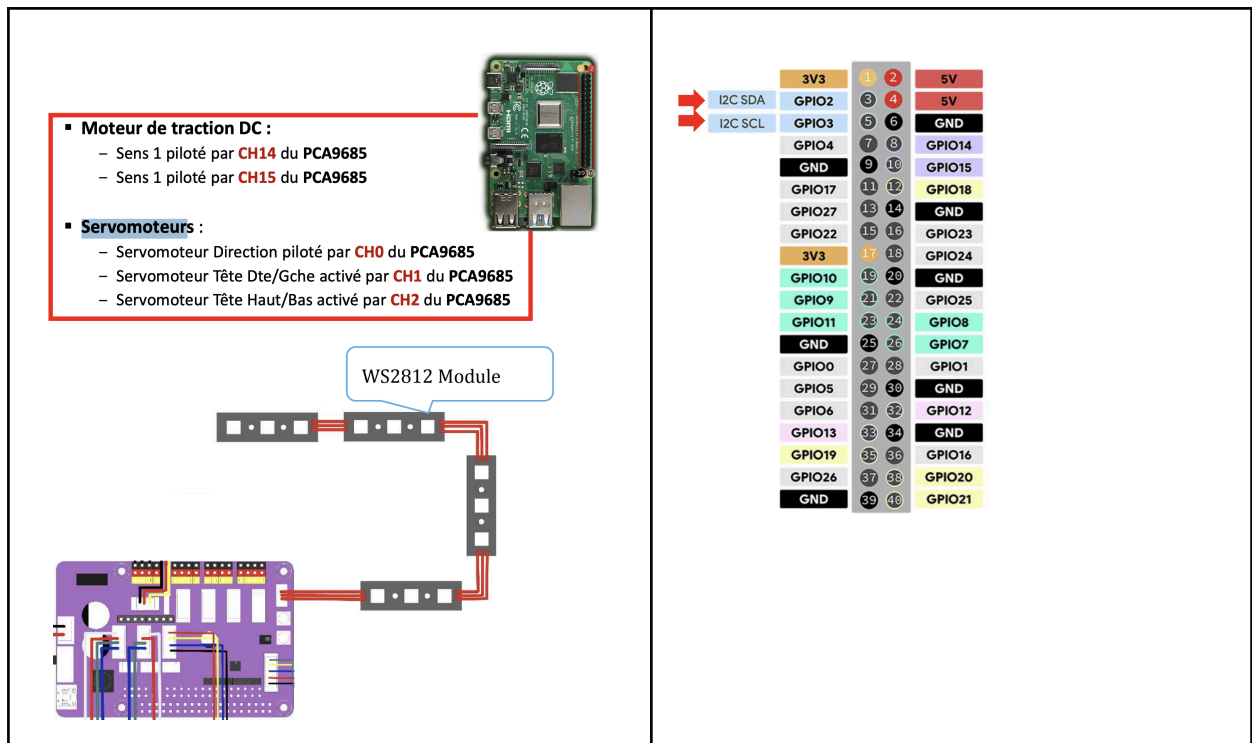
https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache10.py

3. Schémas matériels



Si un sensor émet de la lumière qui se réfléchit sur la surface, il renverra 1 comme valeur, sinon 0.





4. Capture d'écran du programme complet

```

taches > tache10.py > ...
1 import threading
2 import time
3 import smbus
4 import tache3 as t3
5 import tache9 as t9
6
7 class ADS7830(object):
8     def __init__(self):
9         self.cmd = 0x84
10        self.bus = smbus.SMBus(1)
11        self.address = 0x48
12
13    def analogRead(self, chn):
14        return self.bus.read_byte_data(
15            self.address,
16            self.cmd | (((chn << 2 | chn >> 1) & 0x07) << 4)
17        )
18
19 if __name__ == "__main__":
20     try:
21         robot = t9.RobotController()
22         threading.Thread(target=robot.run, daemon=True).start()
23         time.sleep(1)
24
25         adc = ADS7830()
26         controller = t3.ServoController()
27
28         baseline = sum(adc.analogRead(1) for _ in range(20)) / 20
29
30         current_angle = 100
31         controller.set_angle(0, current_angle)
32
33         was_en_marche = robot.en_marche
34
35         try:
36             while True:
37                 ecart = adc.analogRead(1) - baseline
38

```



```

38
39
40     if ecart < -5:
41         current_angle = max(130, current_angle - 5)
42     elif ecart > 5:
43         current_angle = min(60, current_angle + 5)
44
45     controller.set_angle(0, current_angle)
46
47     # Détecte l'arrêt sur obstacle
48     if was_en_marche and not robot.en_marche:
49         print("Obstacle détecté, recul de 30cm")
50         robot.mc.drive_ramp(-t9.RobotController.VITESSE_MARCHE, ramp_time=2)
51         time.sleep(1.5)
52         robot.mc.drive_ramp(0.0, ramp_time=0.1)
53         controller.set_angle(0, 100)
54         time.sleep(2)
55         robot.demarrer()
56
57     was_en_marche = robot.en_marche
58     time.sleep(0.05)
59
60     finally:
61         controller.set_angle(0, 100)
62         controller.deinit()
63
64 except KeyboardInterrupt:

```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

L'exécution de cette tâche s'est déroulée sans réelle difficulté car elle s'appuyait directement sur les tâches 8 et 9. (La tâche 8 nous avait déjà permis de comprendre le fonctionnement des capteurs de lumière) Le travail principal était juste d'ajuster les valeurs pour que le robot tourne correctement en fonction des valeurs détectées.

6. Références et liens vers les bibliothèques utilisées

<https://docs.python.org/3/library/threading.html>

<https://docs.python.org/3/library/time.html>

<https://smbus.org/specs/>

7. Conclusion

Ainsi, cette tâche exécute parfaitement les consignes. Le robot détecte efficacement les obstacles et effectue la marche arrière avec les lumières dans le cas où il détecte un obstacle, comme on peut le voir sur la vidéo de présentation ci-dessous.

Vidéo de présentation:

https://youtube.com/shorts/3Usg6DYAMNo?is=7uOdzdIBy_SWKqAR

Tâche 11: Suivi de ligne

1. Présentation du scénario de test

L'objectif de cette tâche est de faire suivre au robot une ligne noire tracée sur fond blanc, en utilisant ses capteurs de suivi de ligne infrarouge. Ces capteurs détectent la lumière réfléchie par la surface : une surface blanche renvoie beaucoup de lumière (valeur 1), tandis qu'une surface noire l'absorbe (valeur 0). En comparant les valeurs des capteurs gauche et droit, le robot peut déterminer sa position par rapport à la ligne et ajuster sa trajectoire en conséquence.

Le robot démarre en marche avant lorsque l'on envoie le caractère 'M' et s'arrête instantanément si on envoie 'A' ou 'a'. En mode marche, il suit la ligne noire en corrigeant sa direction selon les lectures des capteurs, tout en maintenant la détection d'obstacles active (distance seuil : 20 cm par défaut). Si un obstacle est détecté à moins de 20 cm, il s'arrête avec les feux de détresse (1 s), recule avec des « Bip Bip » (30 cm), fait une pause (2 s), puis reprend le suivi de ligne.

2. Accès au fichier contenant le programme de test

https://github.com/hezaqqq/adeept_picar-b2/blob/master/taches/tache11.py

3. Schémas matériels

Tracking module:

- Left line IR sensor is connected to **GPIO 22**
- Middle line IR sensor is connected to **GPIO 27**
- Right line IR sensor is connected to **GPIO 17**

Ultrasonic sensor:

- Trig pin of the ultrasonic sensor: **GPIO 23**
- Echo pin of the ultrasonic sensor: **GPIO 24**

I2C COM Protocol (**GPIO 2** and **GPIO 3**)

- Servo motors are controlled by the PCA9685 chip:
Interfacing to Raspberry Pi is done using I2C COM protocol with **address 0x5F**
- **MPU6050** uses I2C COM protocol with **address 0x68**
- **OLED screen** uses I2C COM protocol with **address 0x3C**



	3V3	1	2	5V
I2C SDA	GPIO2	3	4	5V
I2C SCL	GPIO3	5	6	GND
	GPIO4	7	8	GPIO14
	GND	9	10	GPIO15
	GPIO17	11	12	GPIO18
	GPIO27	13	14	GND
	GPIO22	15	16	GPIO23
	3V3	17	18	GPIO24
	GPIO10	19	20	GND
	GPIO9	21	22	GPIO25
	GPIO11	23	24	GPIO8
	GND	25	26	GPIO7
	GPIO0	27	28	GPIO1
	GPIO5	29	30	GND
	GPIO6	31	32	GPIO12
	GPIO13	33	34	GND
	GPIO19	35	36	GPIO16
	GPIO26	37	38	GPIO20
	GND	39	40	GPIO21

4. Capture d'écran du programme complet

```
import tache9 as t9
import tache6 as t6
import tache3 as t3
import threading
import time

ANGLE_CENTER = 100
ANGLE_MIN = 60
ANGLE_MAX = 140

if __name__ == "__main__":
    try:
        robot = t9.RobotController()
        threading.Thread(target=robot.run, daemon=True).start()
        time.sleep(1)

        linecap = t6.LineFollower()
        controller = t3.ServoController()

        current_angle = ANGLE_CENTER
        controller.set_angle(0, current_angle)
        was_en_marche = robot.en_marche
        ligne_perdue_ts = None
        angle_avant_perte = ANGLE_CENTER

        robot.demarrer()
```

```

while True:
    r = linecap.right.value
    m = linecap.middle.value
    l = linecap.left.value

    if r == 0 and m == 1 and l == 0:
        current_angle = ANGLE_CENTER
    elif r == 1 and m == 0 and l == 0:
        current_angle += 15
    elif r == 0 and m == 0 and l == 1:
        current_angle -= 15
    elif r == 1 and m == 1 and l == 0:
        current_angle += 5
    elif r == 0 and m == 1 and l == 1:
        current_angle -= 5
    elif r == 1 and m == 1 and l == 1:
        current_angle = ANGLE_CENTER

    elif r == 0 and m == 0 and l == 0:
        angle_avant_perte = current_angle
        if ligne_perdue_ts is None:
            ligne_perdue_ts = time.time()

        elapsed = time.time() - ligne_perdue_ts

        # Angle inversé : symétrique par rapport au centre
        angle_recul = ANGLE_CENTER + (ANGLE_CENTER - angle_avant_perte)
        current_angle = angle_recul

        if robot.en_marche:
            robot.arreter()
            controller.set_angle(0, current_angle)
            robot.mc.drive_ramp(-t9.RobotController.VITESSE_MARCHE, ramp_time=elapsed+0.1)
        ligne_perdue_ts = None # reset pour retenter

    robot.demarrer()

```

```

else:
    ligne_perdue_ts = None

# Sécurité bornes servo
current_angle = max(ANGLE_MIN, min(ANGLE_MAX, current_angle))
controller.set_angle(0, current_angle)

# Reprise après obstacle
if was_en_marche and not robot.en_marche:
    if r != 0 or m != 0 or l != 0:
        print("Obstacle détecté – reprise dans 2s")
        time.sleep(2)
        robot.demarrer()

was_en_marche = robot.en_marche
time.sleep(0.05)

except KeyboardInterrupt:
    pass

finally:
    controller.set_angle(0, ANGLE_CENTER)
    controller.deinit()
    robot.arreter()
    robot.desactiver_feux()
    robot.mc.destroy()

```

5. Retour d'expérience sur les difficultés rencontrées et les solutions mises en œuvre

L'exécution de cette tâche s'est appuyée sur les tâches précédentes, notamment la tâche 9. La principale difficulté résidait dans le calibrage des vitesses de correction : des virages trop brusques faisaient perdre la ligne au robot, tandis que des corrections trop douces entraînaient des dérives. Il a fallu ajuster les valeurs de vitesse moteur gauche/droit pour obtenir un suivi fluide, en particulier dans les virages en angle droit et les courbes sinusoïdales tels que présentés dans le cahier des charges.

6. Références et liens vers les bibliothèques utilisées

<https://docs.python.org/3/library/threading.html>

<https://docs.python.org/3/library/time.html>

<https://smbus.org/specs/>

7. Conclusion

Ainsi, cette tâche exécute parfaitement les consignes et réalise le circuit de validation en 2 minutes. Le robot suit le circuit représenté par une ligne noire, continue même lorsqu'il y a une petite partie blanche dans le circuit, pivote correctement sur les angles droits et les courbes sinusoïdales.

Vidéo de présentation:

<https://youtube.com/shorts/TRXWyTtigqw?is=MANXKBQxqQfwAVGn>